Artificial and Computational Intelligence

Assignment – II

Name of Student	BITS ID	Contribution
Mallidi Akhil Reddy	2024TM93056	100%
Ashwarya Anupam	2024TM93057	100%
Vaidya Rucha Sandeep	2024TM93058	100%
Shivam Prabhakar	2024TM93059	100%
J Rakesh	2024TM93061	100%

PART – A Gaming Catch-Up with Numbers

➢ Game Explanation

- ❖ There are two players – P1 and P2.
- ❖ Numbers available at the start: {1, 2, 3, …, n}.
- ❖ N is agreed upon at the start.
- ❖ Players take turns picking one or more numbers so that their running sum equals or exceeds the opponent's total.
- ❖ Each chosen number is removed from the set.
- ❖ The game continues until all numbers are used.
- ❖ The player with the higher final total wins.

➢ Min-Max Algorithm

- ❖ The Min-Max algorithm is a classic Artificial Intelligence (AI) technique used for two-player, turn-based games.
- ❖ The algorithm helps the computer (AI player) make the best possible move by predicting all possible future moves and their outcomes.
- ❖ The Min-Max algorithm works using a game tree structure:
- ❖ Each node in the tree represents a possible game state (for example, which numbers are left and current totals).
- ❖ From each node, the algorithm explores all possible moves a player can make.
- ❖ For each move, it calculates the resulting scores (using an evaluation function).
- ❖ The algorithm recursively explores the future moves of both players — one as MAX and the other as MIN.
- ❖ Finally, the algorithm backtracks through the tree, choosing:
- ❖ The maximum value when it's the MAX player's turn.
- ❖ The minimum value when it's the MIN player's turn.

➢ Code Flow

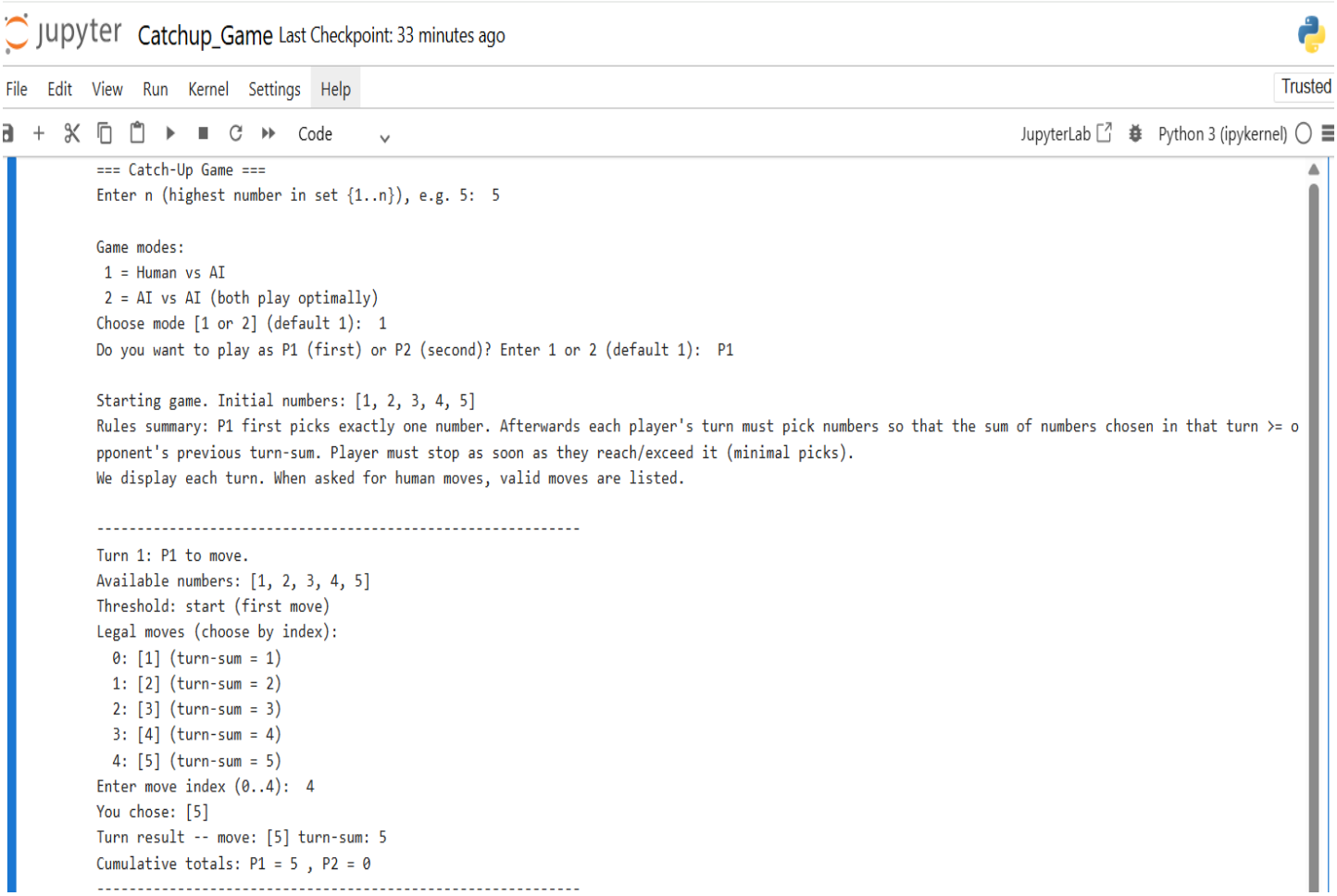
- ❖ Initialize n and create list [1...n]

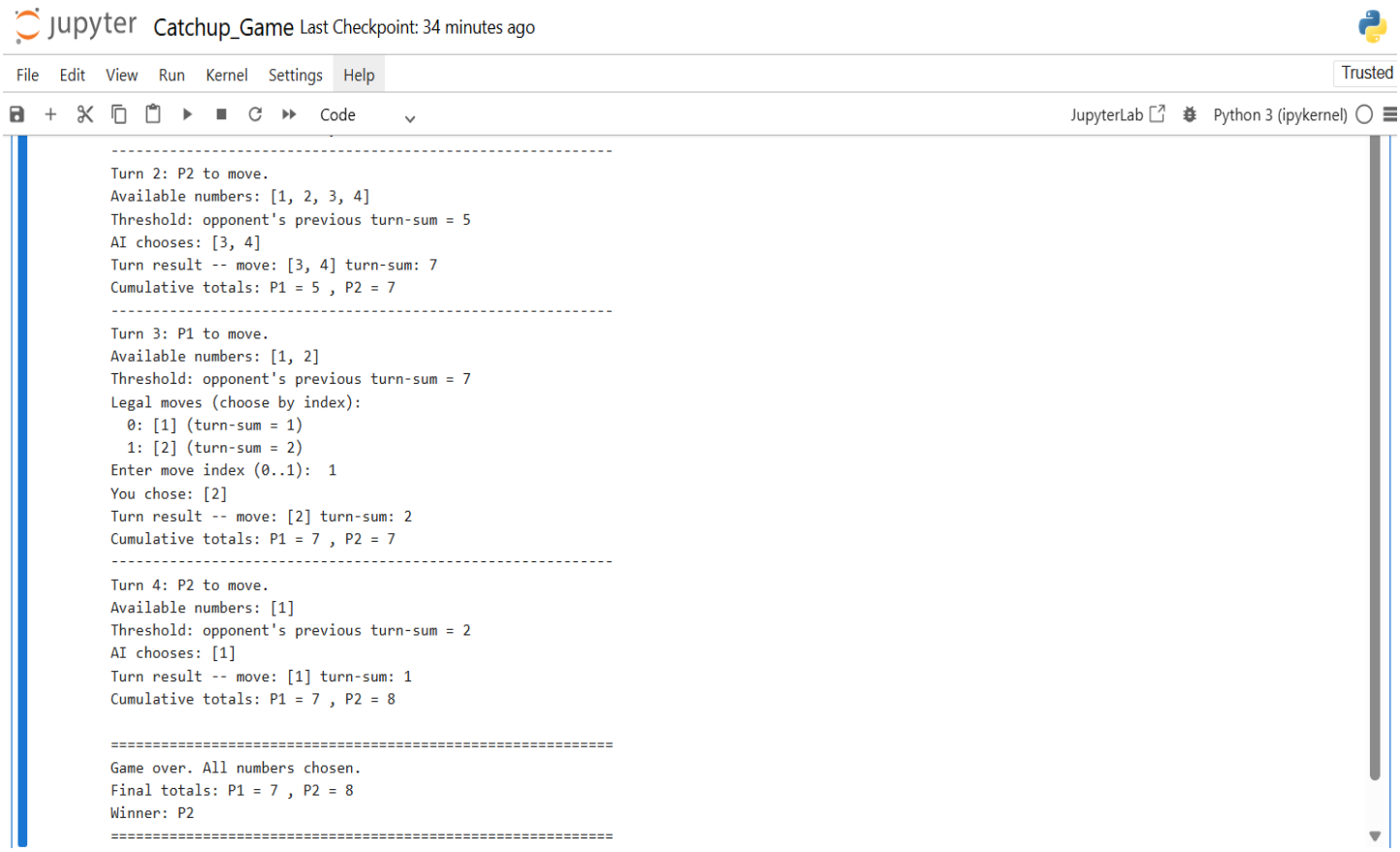
- Initialize totals: P1_total = 0, P2_total = 0
- While numbers remain:
- Show available numbers
- P1 (Max) chooses move using minimax
- Update totals and remove chosen numbers
- Display step
- P2 (Min) chooses move using minimax
- Update totals and remove chosen numbers
- When list empty:
- Compare totals and print winner

➢ Screenshots

Here are the execution step by step

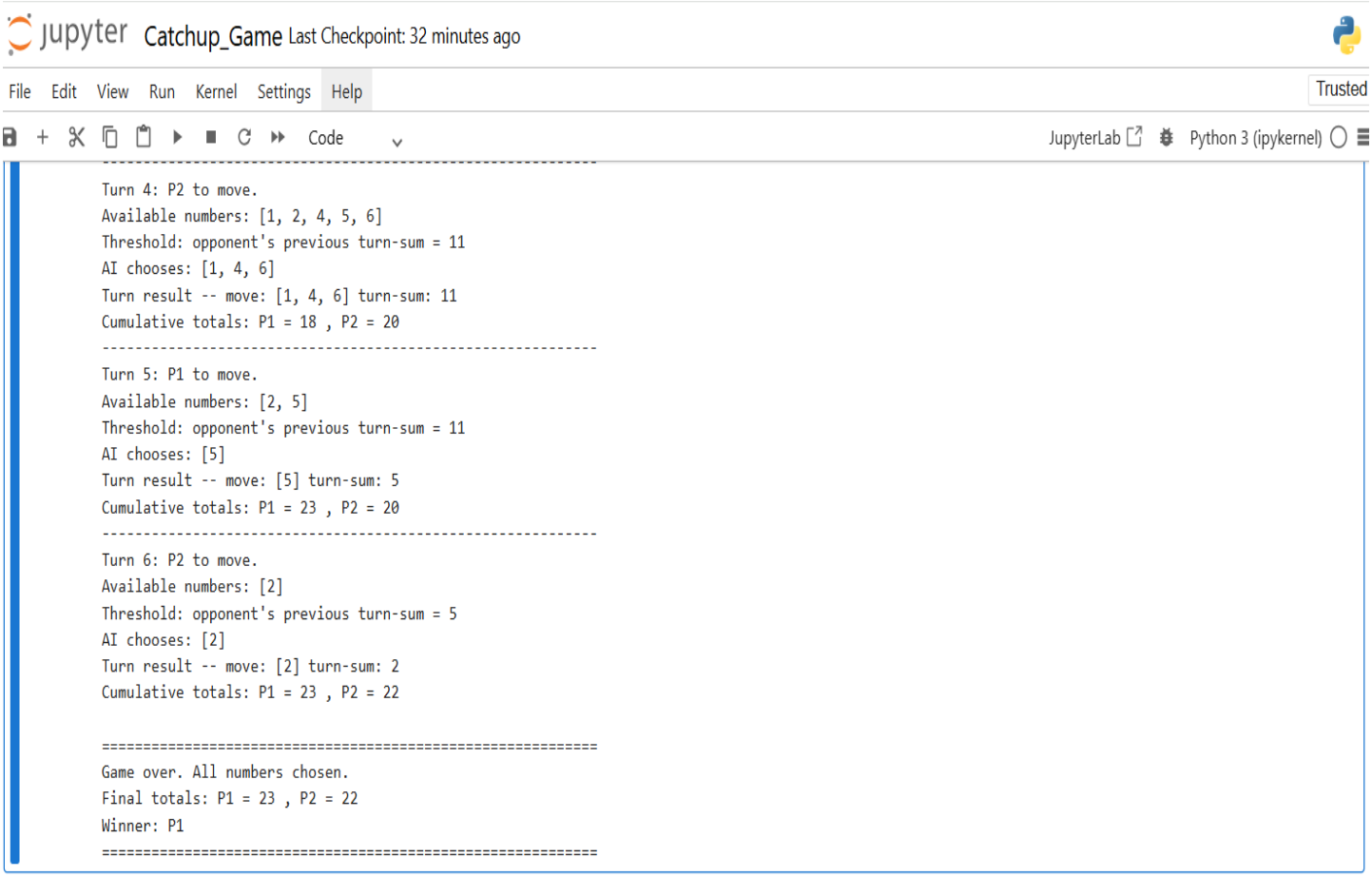
- n=5.
- Game mode = Human vs AI.
- Human is P1





Here,

- n=9.
- AI vs AI



PART-B Logic - Water Resource Prediction Decision Tree

➢ Problem Explanation – Overview

The task is to implement a decision tree system that recommends the most suitable water resource for a community based on geographical and environmental factors. The system uses logic-based reasoning to evaluate multiple attributes and determine whether rainfall, river water, lake water, or groundwater would be the optimal choice.
Input Parameters

The system considers the following factors:

1. Distance from Lake (km) - How far the location is from the nearest lake
2. Distance from River (km) - How far the location is from the nearest perennial river
3. Rainfall Intensity (mm/month) - The amount of rainfall the location receives
4. Sandy Aquifer Presence (yes/no) - Whether the location has a sandy aquifer suitable for groundwater extraction
5. Distance from Beach (km) - How far the location is from the nearest beach

Water Resource Options
The system can recommend one of four water sources:

- Lake - Surface water from a nearby lake
- River - Water from a perennial river
- Rain - Harvested rainwater
- Groundwater - Water extracted from underground aquifers

➢ Decision Tree Logic

The decision tree follows a hierarchical structure with multiple decision points:
Decision Flow

1. First Check: Lake Distance
    - If distance from lake < 10 km → Recommend Lake
    - Otherwise, proceed to next check
2. Second Check: River Distance (if lake distance ≥ 10 km)
    - If distance from river < 8 km:
        - Check rainfall intensity:
            - If rainfall ≥ 200 mm → Recommend Rain
            - If rainfall < 200 mm → Recommend River
    - If distance from river ≥ 8 km, proceed to next check
3. Third Check: Rainfall Intensity (if lake ≥ 10 km AND river ≥ 8 km)
    - If rainfall ≥ 150 mm → Recommend Rain
    - If rainfall < 150 mm, proceed to next check
4. Fourth Check: Sandy Aquifer Presence (if rainfall < 150 mm)
    - If sandy aquifer = YES:
        - Check beach distance:
            - If beach distance < 5 km:
                - Check river distance again:
                    - If river distance < 20 km → Recommend River
                    - If river distance ≥ 20 km → Recommend Rain
            - If beach distance ≥ 5 km → Recommend Groundwater
    - If sandy aquifer = NO:
        - Check lake distance again:
            - If lake distance < 14 km → Recommend Lake
            - If lake distance ≥ 14 km → Recommend Rain

➢ Implementation Approaches

✓ *Python Implementation*

The Python code implements the decision tree using nested conditional statements:
Key Features:

- Interactive input collection with validation
- Sequential evaluation of conditions
- Immediate return upon reaching a decision
- User-friendly prompts and output

❖ Code Structure:

- get_input(): Helper function for user input
- decision_tree(): Main function implementing the logic

-Evaluates conditions in hierarchical order
- Returns recommended water source
- Prints recommendation to console

❖ Logic Flow:

1. Prompt user for lake distance
2. If lake is close enough, recommend lake
3. Otherwise, collect more information progressively
4. Each decision point narrows down options
5. Final recommendation is displayed

❖ Prolog Implementation

The Prolog implementation uses logical rules and pattern matching:
Key Features:

- Declarative logic programming approach
- Rule-based decision making
- Cut operators (!) to prevent backtracking
- Predicate-based architecture

❖ Code Structure:

- main: Entry point that calls process
- process: Collects all inputs and calls decision predicates
- prompt: Helper predicate for user interaction
- decide_water_resource: Multiple clauses implementing decision logic

❖ Logic Flow:

1. Collect all inputs upfront
2. Attempt to match against rule clauses in order
3. First matching clause succeeds and returns result
4. Cut operators ensure only one solution is found

❖ Algorithm Explanation

- ***Decision Tree Algorithm***

A decision tree is a supervised learning algorithm used for both classification and decision-making tasks. It works by:

1. Starting at the root node with all available data
2. Splitting data based on attribute values at each node
3. Creating branches for each possible outcome
4. Continuing recursively until reaching leaf nodes (final decisions)

Advantages:

- Easy to understand and interpret
- Handles both numerical and categorical data
- Requires minimal data preprocessing
- Can model complex relationships

In This Context:

- Root node: Lake distance
- Internal nodes: Various distance and rainfall checks
- Leaf nodes: Final water source recommendations
- Branches: Binary or multi-way splits based on threshold values
-

Example Walkthroughs

Example 1: Location with Close Lake

Input:

- Lake distance: 8 km
- River distance: (not asked)
- Rainfall: (not asked)
- Sandy aquifer: (not asked)
- Beach distance: (not asked)

Decision Path:

1. Check lake distance: 8 < 10
2. Recommendation: Lake

```
Welcome to the Water Resource Decision Tree System!
Enter distance from lake (km): 8
Recommended water source: Lake
```

Reasoning: Since the lake is very close (< 10 km), it's the most convenient and cost-effective option.

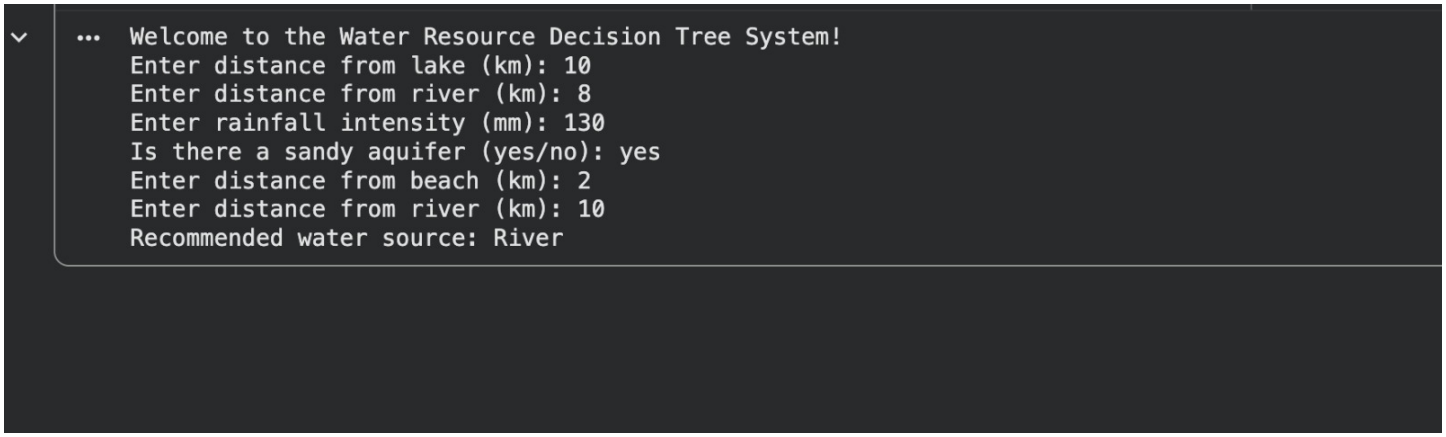
Example 2: Moderate Distance with High Rainfall

Input:

- Lake distance: 10 km
- River distance: 8 km
- Rainfall: 130 mm
- Sandy aquifer: yes
- Beach distance: 2 km
- River distance (second check): 10 km

Decision Path:

1. Check lake distance: 10 ≥ 10, continue
2. Check river distance: 8 ≥ 8, continue
3. Check rainfall: 130 < 150, continue
4. Check sandy aquifer: yes, continue
5. Check beach distance: 2 < 5, continue
6. Check river distance: 10 < 20
7. Recommendation: River



Reasoning: Although initially the river seemed far, upon deeper evaluation with aquifer and beach proximity, the river is still accessible enough to be the best option.

❖ Conclusion

This Part B assignment demonstrates the application of decision trees in a practical water resource management scenario. Both Python and Prolog implementations achieve the same logical decision-making but showcase different programming paradigms:

- Python offers an intuitive, step-by-step approach familiar to most programmers
- Prolog provides a natural way to express logical rules and constraints

The decision tree effectively models expert knowledge about water resource selection, considering multiple environmental and geographical factors to make informed recommendations. This type of system could be extended with machine learning to optimize thresholds based on real-world data or enhanced with additional factors like water quality, cost, and infrastructure requirements.